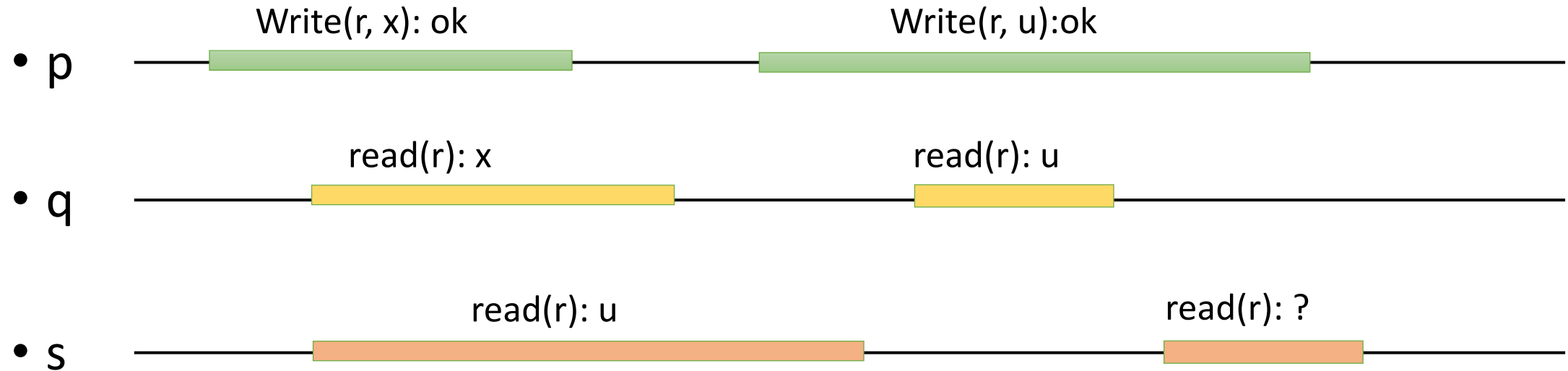

Byzantine Shared Memory

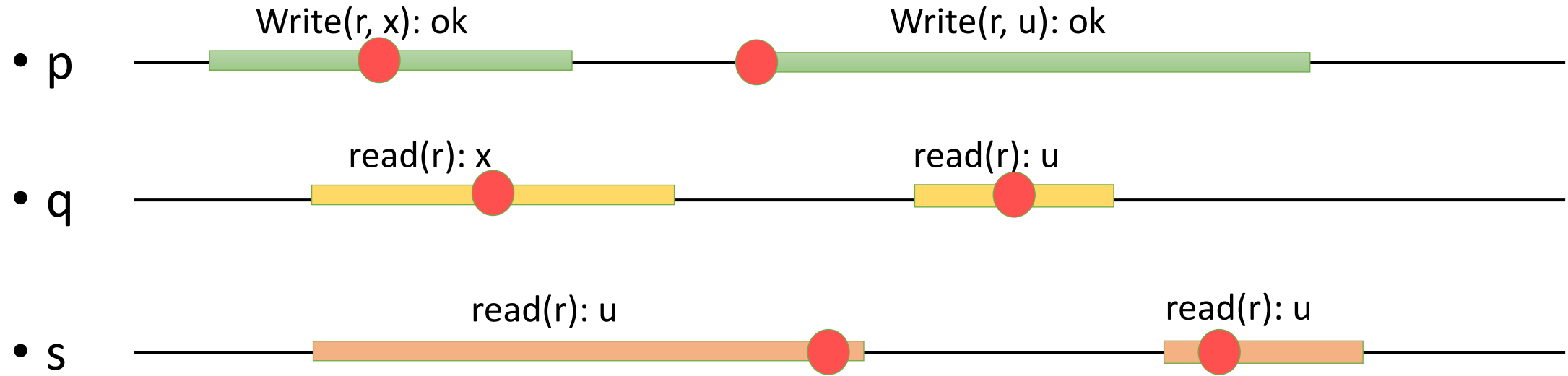
Mohsen Lesani

Semantics of Memory



- **Safe:** Every read not concurrent with a write returns the most recently written value.
- **Regular:** Safe, and any read concurrent with a write returns either the most recently written value or the concurrently written value. The process s may read x or u.
- **Atomic** (linearizable): Regular, and all read and write operations occur atomically. The process s must read u.

Linearizability



- Every operations appears to execute atomically at its linearization point ● which lies in real time between the invocation and the completion.

(1,N) Regular Register (onrr)

- Events

- Request $\langle \text{onrr}, \text{Read} \rangle$
 - Invokes a read operation on the register
- Request $\langle \text{onrr}, \text{Write} \mid v \rangle$
 - Invokes a write operation with value v
- Indication $\langle \text{onrr}, \text{ReadReturn} \mid v \rangle$
 - Completes a read operation, returning value v
- Indication $\langle \text{onrr}, \text{WriteReturn} \rangle$
 - Completes a write operation

- Properties

- ONRR1 (Liveness): If a correct process invokes an operation, then the operation eventually completes.
- ONRR1 (Validity): A read returns the last value written or the value written concurrently.
 - Only one process can possibly write.

Registers in Byzantine model

- Up to f processes may be Byzantine, including the reader
- Writer process is always correct
- Specification of
 - $(1, N)$ safe Byzantine register (bonsr) and
 - $(1, N)$ regular Byzantine register (bonrr)
directly follows from $(1, N)$ regular register

Implementations

- Algorithms must eliminate wrong values returned by Byzantine processes
- Two approaches for elimination
 - Masking by sufficiently many correct values
 - Algorithm "Masking Quorum" for Byzantine safe register
 - Authentication of correct values with digital signatures
 - Algorithm "Authenticated Data" for Byzantine regular register

Byzantine Masking Quorum

Implements bonsr, uses **al**, **abeb** ($N > 4f$), writer is **w**

$(ts, val) := (0, \perp)$; $wts := 0$; $rid := 0$

upon <bonsr, Write | **v**> **do**

$wts := wts + 1$

$acklist := [\perp]^N$

trigger <**abeb**, Broadcast | [WRITE, wts , **v**]>

upon <**abeb**, Deliver | **w**, [WRITE, ts' , v']>

if $ts' > ts$ **then**

$(ts, val) := (ts', v')$

trigger <**al**, Send | **w**, [ACK, ts']>

upon <**al**, Deliver | **q**, [ACK, wts]> **do**

$acklist[q] := \text{true}$

if $\#(acklist) > (N+2f)/2$ **then**

trigger <bonsr, WriteReturn>

Differences are in this color

Byzantine Masking Quorum

upon <bonsr, Read> **do**

rid := rid + 1

readlist := $[\perp]^N$

trigger <abeb, Broadcast | [READ, rid]>

upon <abeb, Deliver | p, [READ, r]> **do**

trigger <al, Send | p, [VALUE, r, ts, val]>

upon <al, Deliver | q, [VALUE, rid, ts', v']> **do**

readlist[q] := (ts', v')

if #(readlist) > $(N+2f)/2$ **then**

v := byz-highest-val(readlist) // filter and extract value

trigger <bonsrr, ReadReturn | v>

byz-highest-val()

eliminates all values occurring f or fewer times (not to be fooled by Byzantine processes)

returns survivor value with highest timestamp

or special value $\perp$ if no such value exists.

Differences are in this color

Byzantine Masking Quorum

- The algorithm may return $\perp$ during concurrent writes.
 - Implements safe register on domain with $\{\perp\}$
- Let's assume $N = 4f+1$
- Without concurrent write operations
 - The last write operation has touched at least $3f+1$ processes.
 - Among them, at least $2f+1$ are correct.
 - The read operation obtains value from at least $3f+1$ processes.
 - Among them, at least $2f+1$ are correct.
 - The total number is at least $4f+2$. There are $3f+1$ correct processes. Thus, there are at least $f+1$ correct processes in the intersection that contain last written timestamp/value pair.

Validity

Authenticated Data Byzantine Quorum

Implements bonrr, uses *al*, *abeb*, signatures ($N > 3f$), writer is *w*

$(ts, val, s) := (0, \perp, \perp); wts := 0; rid := 0$

upon <bonrr, Write | *v*> **do**

$wts := wts + 1; s := \text{sign}(w, \text{WRITE} \mid w \mid wts \mid v)$

$\text{ack-list} := [\perp]^N$

trigger <*abeb*, Broadcast | [WRITE, *wts*, *v*, *s*]>

upon <*abeb*, Deliver | *w*, [WRITE, *ts'*, *v'*, *s'*]> **do**

if *ts'* > *ts* **then**

$(ts, val, s) := (ts', v', s')$

trigger <*al*, Send | *w*, [ACK, *ts'*]>

upon <*al*, Deliver | *q*, [ACK, *wts*]> **do**

$\text{ack-list}[q] := \text{true}$

if $\#(\text{acklist}) > (N+f)/2$ **then**

trigger <bonsr, WriteReturn>

Differences are in this color

Authenticated Data Byzantine Quorum

upon <bonrr, Read> **do**

rid := rid + 1

read-list := $[\perp]^N$

trigger <abeb, Broadcast | [READ, rid]>

upon <abeb, Deliver | p, [READ, r]> **do**

trigger <al, Send | p, [VALUE, r, ts, val, s]>

upon <al, Deliver | q, [VALUE, rid, ts', v', s']> **do**

if **verifysig**(w, WRITE | w | ts' | v', s') **then**

read-list[q] := (ts', v')

if #(read-list) > (N+f)/2 **then**

v := **highest-val**(read-list) // value with highest ts

trigger <bonrr, ReadReturn | v>

Differences are in this color

Authenticated Data Byzantine Quorum

- The algorithm uses
 - Digital signatures issued by writer
 - Byzantine quorums
- Otherwise, exactly the same as the Majority Quorum algorithm
- Digital signatures prevent a Byzantine process from sending a message with an arbitrary timestamp and value.
- A Byzantine process may include a signed value with an outdated timestamp. However, the latest value is recovered from Byzantine quorum intersection.